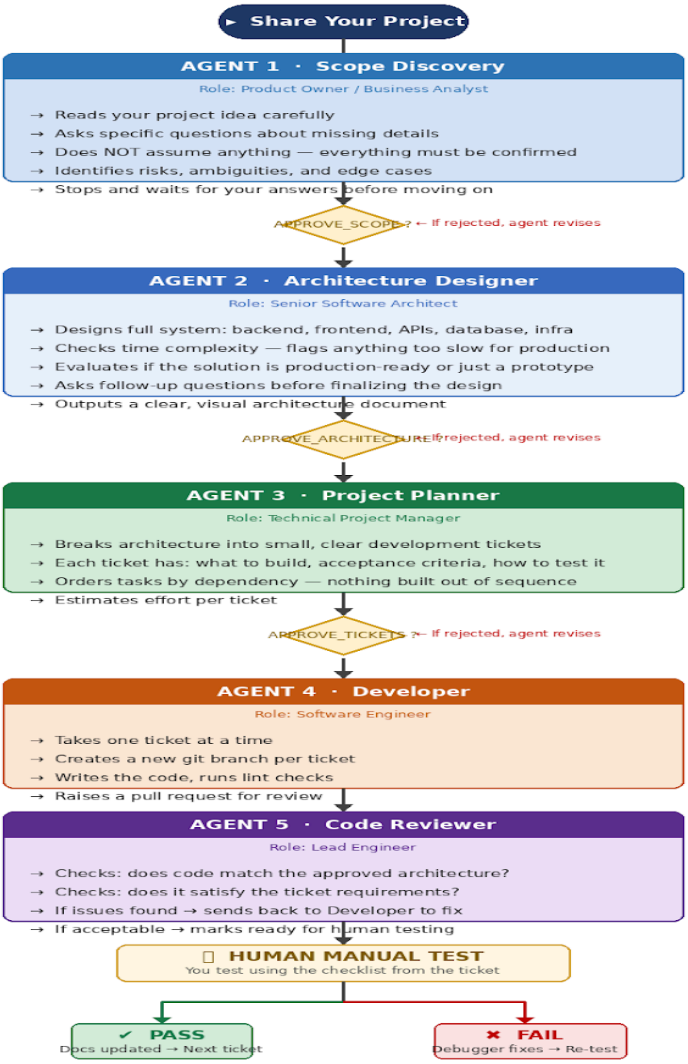
AI-Powered Multi-Agent Development System

A proposal to automate our software development pipeline using AI agents — with full human control at every critical decision point.

How the System Works — Pipeline Overview

Below is the full flow of the system. Think of it like an assembly line where each station has a specific job, and nothing moves to the next station until the current one is done and checked.

AI Multi-Agent System — Pipeline Flow



The graph does not move forward unless the human approves. Built with LangGraph.

The Agents — What Each One Does

Here is a detailed breakdown of each agent in the pipeline and exactly what it is responsible for.

Agent 1 · Scope Discovery *(acts like a Product Owner)*

→ Reads your project description from start to finish
→ Identifies every part that is unclear or missing
→ Asks you a structured list of questions — one by one, clearly
→ Does not assume anything. If it does not know, it asks
→ Flags risks early: things that could go wrong if left unclear

⏸ **Stops here:** *Waits for your answers. Does not proceed until all questions are answered.*

▼ APPROVE_SCOPE ▼

Agent 2 · Architecture Designer *(acts like a Senior Architect)*

→ Reads the confirmed scope and designs the full system
→ Covers: backend, frontend, APIs, database schema, infrastructure
→ Checks time complexity — if something is too slow for production, it flags it and suggests a better approach
→ Evaluates whether this design is production-ready or just suitable for a demo
→ Asks follow-up questions if any architectural decision depends on your preference
→ Outputs a clear architecture document you can actually understand

⏸ **Stops here:** *Waits for you to review the architecture. Nothing gets built until you confirm it.*

▼ APPROVE_ARCHITECTURE ▼

Agent 3 · Project Planner *(acts like a Technical PM)*

→ Takes the approved architecture and turns it into a ticket list
→ Each ticket says: what to build, what counts as done, how to manually test it
→ Tasks are ordered correctly — nothing is built before its dependency is ready
→ Estimates effort so you know how long each piece will take

⏸ **Stops here:** *Waits for you to review and confirm the task list before any coding starts.*

▼ APPROVE_TICKETS ▼

Agent 4 · Developer Agent *(acts like a Software Engineer)*

→ Picks up one ticket at a time — no multitasking, no skipping ahead
→ Creates a new git branch for the ticket
→ Writes the code and runs basic checks (lint, build)
→ Raises a pull request and hands it to the Reviewer

⏸ **Stops here:** *Does not self-merge. All code goes through review first.*

Agent 5 · Code Reviewer *(acts like a Lead Engineer)*

→ Checks if the code actually matches what the architecture says
→ Verifies the ticket acceptance criteria are met
→ If something is wrong or messy, sends it back to the Developer to fix
→ Only marks it ready when the code is genuinely acceptable

⏸ **Stops here:** *Code only reaches human testing after passing this review.*

HUMAN MANUAL TEST GATE

This is where you step in. Using the checklist the Planner wrote for this ticket, you manually test the feature. Then you respond with one of two things:

PASS — feature works as expected. Docs are updated and we move to the next ticket.

FAIL — something is broken. Describe the issue and paste any logs. The Debugger Agent takes it from there, fixes it, and the code goes through review and testing again.

The Three Human Approval Gates

These are the moments in the pipeline where the system fully stops and waits for you. Nothing happens until you say so.

- APPROVE_SCOPE — you confirm the project requirements are correct before any architecture work begins
- APPROVE_ARCHITECTURE — you confirm the system design before any planning or coding begins
- APPROVE_TICKETS — you confirm the task breakdown before the Developer Agent starts writing code
- Manual Test Result — you test each completed ticket and mark it PASS or FAIL

What We Get Out of This

To be direct about the practical benefits:

- Faster turnaround on structured work — requirements, architecture, and planning happen in minutes instead of days
- No assumptions baked into the code — every decision traces back to something we explicitly approved
- Consistent documentation — automatically updated after every ticket, not written at the end when everyone has forgotten what happened
- A proper debugging loop — when something breaks, the system does not just flag it and stop. It fixes it, reviews it, and re-tests it
- We stay in control — the AI cannot ship anything without a human sign-off at every major stage

---